IAM-99: Best Practice Summary

> **Series:** IAM — IAM Administration | **Notebook:** Bonus Summary | **Created:** March 2026 | **Last Updated:** 11/08/2026

Overview

This notebook distills every actionable best practice from the IAM series (IAM-01 through IAM-12) into a single reference. Each practice is definitive: it states exactly what to configure, at what priority, and in which category. Use this as a pre-deployment checklist and ongoing governance audit sheet.

Table of Contents

1. Authentication and SSO
2. Group Architecture
3. Policy Design
4. Boundaries and Data Partitioning
5. User Lifecycle and Provisioning
6. Tokens and Service Accounts
7. Audit and Compliance
8. Multi-Environment and Governance
9. Troubleshooting and Operations

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Gen3 IAM enabled
Prior Reading	Complete **IAM-01** through **IAM-12** or use this as a reference while working through the series
Permissions	`account-viewer` or higher to audit existing IAM configuration

Priority Legend

Priority	Meaning
Critical	Must implement before production. Security risk if omitted.
Recommended	Should implement for enterprise-grade operations.
Optional	Beneficial for mature organizations or specific compliance needs.

1. Authentication and SSO

Source: IAM-01, IAM-02

#	Best Practice	Recommended Setting/Value	Priority	Category
1	Configure SSO before creating groups or policies	SAML 2.0 IdP integration in Account Management > Identity & access management > SAML configuration	**Critical**	Auth
2	Sign the entire SAML response, not just the assertion	IdP configuration: sign the whole SAML message. Dynatrace: *"It isn't sufficient to sign just the assertion part."*	**Critical**	Auth
3	Set NameID format to email	IdP SAML app: Name ID format = EmailAddress	**Critical**	Auth
4	Include groups attribute in SAML assertion	IdP attribute statement: name = `groups`, value = user group memberships	**Critical**	Auth
5	Enforce MFA at the IdP level for all Dynatrace users	IdP sign-on policy: Require MFA for the Dynatrace SAML application	**Critical**	Auth
6	Use phishing-resistant MFA factors	Factor priority: WebAuthn > TOTP > Push > SMS. Disable SMS if possible	**Recommended**	Auth
7	Maintain at least one local admin account	Local Dynatrace account (not SSO-dependent) with `account-user-management`, stored in enterprise vault	**Critical**	Auth
8	Test both SP-initiated and IdP-initiated login	Verify: Dynatrace login redirects to IdP AND IdP portal tile lands in Dynatrace	**Recommended**	Auth
9	Do not enable assertion encryption	Dynatrace's IdP requirements list *"No assertion encryption"* — protect the assertion with signing (#2) and TLS instead	**Recommended**	Auth

> **Sources:** SAML (DT docs).

2. Group Architecture

Source: IAM-03, IAM-06

#	Best Practice	Recommended Setting/Value	Priority	Category
10	Use the Matrix group pattern for enterprises (>50 users)	Groups = `<scope>-<team>-<role>` (e.g., `dt-checkout-admins`, `dt-checkout-viewers`)	**Recommended**	Groups
11	Prefix all Dynatrace groups with `dt-`	Naming: `dt-<team>-<role>`, lowercase, hyphens only, under 50 characters	**Critical**	Groups
12	Map IdP groups explicitly to Dynatrace groups you created	Create the group in Account Management first, then add the IdP group names as its SAML group attribute values. Adding SAML values to an existing LOCAL group removes its current members — plan the switch	**Critical**	Groups
13	Assign all access via groups, never directly to users	Direct user assignment only for break-glass, time-limited scenarios	**Critical**	Groups

#	Best Practice	Recommended Setting/Value	Priority	Category
14	Designate an owner for every group	Group description must include owner name/team and purpose	**Recommended**	Groups
15	Create a default viewer group for all authenticated users	Group `dt-all-viewers` with the Standard User default policy, mapped to IdP all-employees group	**Recommended**	Groups
16	Create a break-glass admin group, empty by default	Group `dt-breakglass-admins` with `account-user-management` + the Admin User default policy. Membership = 0 at rest. Temporary membership < 24h only	**Critical**	Groups
17	Enforce separation of duties via incompatible group pairs	Users must NOT be in both: Production Admins + Development Editors, Security Auditors + Platform Admins, Token Managers + Application Users	**Recommended**	Groups
18	Review admin group membership quarterly	Admin groups: quarterly. Account admins: monthly. Editor groups: semi-annually. Viewer groups: annually (same cadence as #56 and IAM-07)	**Critical**	Groups

3. Policy Design

Source: IAM-04, IAM-10, IAM-97

#	Best Practice	Recommended Setting/Value	Priority	Category
19	Start with default policies, add custom only when needed	Use the Standard User, Pro User and Admin User default policies as baseline. Create custom policies for team-scoped or capability-scoped access	**Recommended**	Policies
20	Layer default + custom policies per group	Group gets Standard User (base access) + custom policy (scoped write). Never rely on custom policy alone for read access	**Recommended**	Policies
21	Use `${bindParam:...}` templated policies for multi-team scoping	One template policy + N bindings instead of N identical policies. Naming: `tpl-<scope>-<permission>`. Bind them through the API or Terraform: *"Currently, group management does not support binding policies with parameters."*	**Recommended**	Policies
22	Apply least privilege: grant minimum required service:resource:action	Each policy statement = `ALLOW <service>:<resource>:<action>`. No wildcards unless role genuinely needs broad access	**Critical**	Policies
22a	Check what a permission does before you grant it	**IAM-97** lists each *Standard User* permission with its scope, risk and basis, and shows how to list the permissions an app's manifest requests. Copies of *Standard User* for readers usually drop `state-management:user-app-states:delete-all` (resets every user's app state) and `email:emails:send` (no condition limits the recipients)	**Recommended**	Policies
23	Use WHERE conditions to scope storage access by security context	`ALLOW storage:logs:read WHERE storage:dt.security_context = "team-name"`	**Critical**	Policies

#	Best Practice	Recommended Setting/Value	Priority	Category
24	Add `storage:bucket-name` conditions only where a bucket already exists for another reason	Lead with `dt.security_context` (#23). Buckets are *"not intended as the primary mechanism for securing or segmenting data access"*; where a bucket exists for retention, cost or compliance, `ALLOW storage:logs:read WHERE storage:bucket-name = "team_logs"` can narrow access to it	**Optional**	Policies
25	Combine bucket + security context for defense in depth where #24 applies	`WHERE storage:bucket-name = "team_logs" AND storage:dt.security_context = "team"`	**Optional**	Policies
26	Version-control all policies via GitOps (Monaco or Terraform)	Store policy YAML in git. PR review before apply. CI/CD deploys to environments	**Recommended**	Policies
27	Test every custom policy in a non-production environment first	Create test group + test user, verify intended access works AND unintended access is blocked	**Critical**	Policies

> **Sources:** Policy templating (DT docs), Partition data (DT docs), IAM policy reference (DT docs).

4. Boundaries and Data Partitioning

Source: IAM-05

#	Best Practice	Recommended Setting/Value	Priority	Category
28	Attach one boundary per policy binding, covering every domain the policy grants	One boundary may combine `storage:dt.security_context IN (...)`, `settings:dt.security_context IN (...)` and `environment:management-zone IN (...)` lines — Dynatrace's own *Kubernetes* boundary example mixes a management-zone and a Grail condition. **Unverified:** the zone line and the `settings:` line both apply to `settings:objects:*`, and Dynatrace does not say whether different condition names on one permission are ANDed — dry-run a pilot user's effective permissions (`effectivepermissions:dry-run`) before relying on the combined shape, or use the split-policy option below. Do not attach a second boundary to the same binding: *"effective statements are calculated for each boundary separately"*, so a domain one boundary does not mention is left unconditional under it. Splitting Gen2 and Gen3 permissions into separate policies, each bound with its own boundary, is also valid and lets MZ retirement remove the Gen2 binding cleanly	**Critical**	**Boundaries**
29	Include "shared" context in team boundaries	`storage:dt.security_context IN ("team-a", "shared", "infrastructure")` for infrastructure visibility	**Recommended**	Boundaries

#	Best Practice	Recommended Setting/Value	Priority	Category
30	Measure security context coverage on the data records, not on entities	Count records without `dt.security_context` per table (logs, spans, events) — query in MZ2POL-06 §3. Entity coverage can read 100% while a large share of logs and events carry none, and records without it are invisible to users scoped only by it. Target: >95% per table	**Critical**	Boundaries
31	Set security context at the source, not with auto-tags	OneAgent `--set-host-tag="dt.security_context=<value>"`, `DT_TAGS` per process, Kubernetes labels/annotations, OTel resource attributes; OpenPipeline enrichment last. Auto-tagging rules *"have no effect on Smartscape on Grail"*	**Recommended**	Boundaries
32	Plan Grail buckets carefully: names are immutable, data cannot move	Naming: `<org>_<datatype>_<retention>` (e.g., `team_a_logs_90d`). Up to 250 custom buckets per environment by default from SaaS 1.346 (80 before). 1 data type per bucket	**Critical**	Boundaries
33	Size log buckets at 1–3 TB/day	Target 1–3 TB/day per log bucket; split other tables above 2 TB/day. No per-bucket cap is documented, but the 500 GB scan limit shrinks the queryable window as a bucket grows; Dynatrace's split guidance varies between pages (IAM-05)	**Recommended**	Boundaries
34	Use Primary Grail Fields for pipeline routing	Route to buckets based on `k8s.cluster.name`, `k8s.namespace.name`, `aws.account.id`, `azure.subscription`, `dt.host_group.id`	**Recommended**	Boundaries
35	Give broad access by leaving bindings unbounded, not by wildcards	Leave admin/platform bindings **unbounded** rather than writing a wildcard; `IN ("*")` matches a zone literally named `*`	**Critical**	Boundaries
35a	Scope lookup files by folder, not by security context	`storage:files:*` accept only `storage:file-path`. Give every team a folder `/lookups/<team>/`. Bind one central read/write policy per team group with a boundary `storage:file-path startsWith "/lookups/<team>/";`. Put shared read on `/lookups/global/` in a separate unbounded policy. Keep both slashes in every prefix (IAM-05 Pattern 8)	**Recommended**	Boundaries
35b	Keep file-path-bounded policies to `storage:files:*` only	A file-path boundary does not restrict any other permission in the same policy. Add `storage:logs:read` there and it becomes unrestricted. Check *Boundary assignments per policy* (10) before sharing one policy across more than 10 teams	**Critical**	Boundaries

Sources: Policy boundaries (DT docs), Tags difference (DT docs), OneAgent primary tags (DT docs), Partition data (DT docs), How to organize your data stored in Grail (DT docs) — *"The default limit per environment is 250 custom buckets."*, Logs bucket strategy best practices (DT docs) — *"Design your bucket strategy so that each bucket receives between 1 TB and 3 TB of log data per day."*, Lookup data in Grail (DT docs). **Derived:** #28 "left unconditional" combines the per-boundary evaluation quote with *"Only conditions added to service configuration will be applied to relative permissions."*

5. User Lifecycle and Provisioning

Source: IAM-06

#	Best Practice	Recommended Setting/Value	Priority	Category
36	Use SCIM for automated user/group sync (>50 users)	IdP SCIM endpoint to Dynatrace. Enable: Create Users, Update Attributes, Deactivate Users, Push Groups	**Recommended**	Lifecycle
37	Know that SAML provisions users just in time	Dynatrace creates users on their first SAML login (*"you don't have to invite users to Dynatrace"*); group membership comes from the SAML group mapping (#12). Use SCIM (#36) when you also need automatic deprovisioning	**Optional**	Lifecycle
38	Immediately revoke all access on employee termination	Same-day: disable in IdP (SCIM auto-deactivates), revoke all API tokens, deactivate OAuth clients, transfer dashboard ownership	**Critical**	Lifecycle
39	Revoke user-created tokens on offboarding	Platform tokens: account admins *"can disable or delete platform tokens created by all users under their account"* (Account Management). Classic access tokens are environment-level — revoke them in each environment's **Access Tokens** page. In community practice, do not assume user deactivation revokes them	**Critical**	Lifecycle
40	Create dedicated groups for external users	Group `external-<company>-<role>` (e.g., `external-acme-consultants`). Apply least-privilege policies with WHERE clauses. Set time-bound access	**Recommended**	Lifecycle
41	Enforce MFA for all external accounts in the IdP	*"Configuring multi-factor authentication (MFA) is not supported by Dynatrace"* — federate external users through an IdP that enforces MFA (#5)	**Critical**	Lifecycle
42	Conduct quarterly access reviews for external users	Review all external user access quarterly. Revoke immediately when engagement ends	**Recommended**	Lifecycle

> **Sources:** SAML (DT docs), Platform tokens (DT docs), Access tokens (DT docs).

6. Tokens and Service Accounts

Source: IAM-06, IAM-12

#	Best Practice	Recommended Setting/Value	Priority	Category
43	One OAuth client per application	Never share clients across apps. Naming: `<app>-<environment>-<purpose>`	**Critical**	Tokens
44	Grant minimum OAuth scopes	Only grant scopes the integration needs. Common: `storage:logs:read`, `storage:metrics:read`, `settings:objects:read`	**Critical**	Tokens

#	Best Practice	Recommended Setting/Value	Priority	Category
45	Rotate OAuth client secrets; do not plan around token lifetime	Client-credentials access tokens expire in 300 s (`expires_in: 300`) and OAuth token expiry is not configurable — fetch a fresh token per run. In community practice, rotate client secrets every 90 days	**Recommended**	Tokens
46	Use descriptive token names	Naming: `<purpose>-<scope>-<owner>` (e.g., `cicd-pipeline-metrics-reader`). Never use generic names	**Recommended**	Tokens
47	Set expiration dates on all API tokens	Every token must have a defined expiry. Default rotation schedule: 90 days	**Critical**	Tokens
48	Store tokens in a secrets manager	Use CyberArk, HashiCorp Vault, AWS Secrets Manager. Never in code, shared environment variables, or wiki pages	**Critical**	Tokens
49	Maintain a service account inventory	Document: owner, purpose, scopes, expiry date, rotation schedule for every OAuth client and API token. Review quarterly	**Recommended**	Tokens
50	Use OAuth2 client credentials for new integrations	Prefer OAuth over API tokens for all new automation. API tokens only for legacy compatibility. Required in Phase 3 environments, where classic API token creation is disabled; the `/apiTokens` endpoints are deprecated in API 1.348 (rollout start 10/12/2026, staged tenant rollout)	**Recommended**	Tokens

> **Sources:** OAuth clients (DT docs), Token concepts (DT docs).

7. Audit and Compliance

Source: IAM-07

#	Best Practice	Recommended Setting/Value	Priority	Category
51	Know each audit trail's retention and export what your framework needs beyond it	Environment audit events on Grail are *"retained for one year"*; account audit log data *"is stored for up to 10 years"*. Retention is not a setting you choose — where a framework requires more (SOX: 7 years, HIPAA: 6 years), export the events to your archive	**Critical**	Audit
52	Monitor failed logins and alert on brute-force patterns	Alert: >5 failed logins from same user in 10 min = Warning. >20 failed logins any user in 10 min = Critical	**Critical**	Audit
53	Alert on all admin actions outside business hours	Alert: any audit event matching admin activity where hour < 8 or hour > 18 = Warning	**Recommended**	Audit
54	Alert on all policy creation, modification, and deletion	Alert: audit events with policy created/modified/deleted = Info severity	**Recommended**	Audit
55	Build a compliance dashboard with five sections	Sections: Authentication trend (line), Failed access by day (bar), User changes (table), Admin activity (table), Token operations (table)	**Recommended**	Audit

#	Best Practice	Recommended Setting/Value	Priority	Category
56	Conduct access reviews at defined frequency by role	Admin groups: quarterly. Editor groups: semi-annually. Viewer groups: annually. Account Admins: monthly	**Critical**	Audit
57	Remove users inactive for 90+ days	Query users with last login >90 days ago. Disable accounts. Transfer asset ownership. Revoke tokens	**Recommended**	Audit
58	Alert on bulk data access patterns	Alert: >1000 queries/hour from a single user or token = Warning (potential exfiltration)	**Optional**	Audit

> **Sources:** Audit logs on Grail (DT docs), Account audit logs (DT docs).

8. Multi-Environment and Governance

Source: IAM-01, IAM-08

#	Best Practice	Recommended Setting/Value	Priority	Category
59	Adopt the hybrid governance model	Central IAM team owns: account-level policies, production access, policy templates, audit. Team leads own: non-prod access, group membership	**Recommended**	Governance
60	Restrict account-level permissions to IAM and platform teams	`account-user-management` : IAM team only. `account-viewer` : limited distribution. No developers with account-level permissions	**Critical**	Governance
61	Separate production and non-production access into distinct groups	Use environment-specific groups (e.g., `dt-prod-checkout-viewers` , `dt-nonprod-checkout-editors`) or environment-scoped policy bindings	**Critical**	Governance
62	Document and drill break-glass procedures	Written procedure: authorized situations, retrieval steps, notification requirements, mandatory credential rotation after use, post-incident review within 24h	**Critical**	Governance
63	Synchronize IAM config across environments via config-as-code	Store groups, policies, boundaries in git. Deploy with Monaco or Terraform. PR > review > test in dev > staging > production	**Recommended**	Governance
64	Require all IAM changes to go through change management	Process: Request (ticket) > Review (IAM admin) > Approval (security lead) > Implementation > Verification > Documentation	**Critical**	Governance
65	Maintain an up-to-date IAM documentation set	Required: Access Policy (annual), Group Catalog (quarterly), Policy Inventory (quarterly), Boundary Definitions (quarterly), Onboarding/Offboarding Guides	**Recommended**	Governance
66	Define RACI matrix for IAM operations	Account Admin = Accountable, IAM Admin = Responsible, Env Admin = Consulted, Auditor = Informed for all IAM activities	**Recommended**	Governance

9. Troubleshooting and Operations

Source: IAM-09, IAM-12

#	Best Practice	Recommended Setting/Value	Priority	Category
67	Follow the 8-step diagnostic methodology for every access issue	1) Authenticate 2) User exists 3) Correct groups 4) Policies grant access 5) Conditions met 6) Boundary allows 7) No conflicts 8) Data exists	**Recommended**	Operations
68	Collect standard information before troubleshooting	Required: user email, exact error message, action attempted, timestamp, recent changes (role, team, IdP)	**Recommended**	Operations
69	Use SAML tracer browser extension for SSO debugging	Check: NameID format = email, groups attribute present, assertion not expired, signature valid	**Recommended**	Operations
70	Define a 4-level escalation path	L1: Team lead. L2: IAM admin (policy/boundary changes). L3: Platform team (IdP/infrastructure). L4: Dynatrace support (product issues)	**Recommended**	Operations
71	Use the IAM API to programmatically validate bindings	GET `/iam/v1/repo/{levelType}/{levelId}/bindings/{policyUuid}` to dump and compare parameter values	**Optional**	Operations
72	Verify security context values match templated policy parameters	DQL: `fetch logs, from:-1h` (and spans, events) summarized `by:{dt.security_context}` — check the record values match `${bindParam:...}` binding values exactly	**Critical**	Operations

---

Summary

This notebook consolidates **75 best practices** across 9 categories from the IAM series (IAM-01 through IAM-12). Use it as:

- **Pre-deployment checklist** -- work through all Critical items before production
- **Quarterly audit sheet** -- verify Recommended items are in place
- **Onboarding reference** -- new IAM administrators start here

Priority Distribution

Priority	Count	Action
Critical	34	Must implement before production
Recommended	36	Implement for enterprise-grade operations
Optional	5	Beneficial for mature or compliance-heavy organizations

Cross-References

Notebook	Focus
IAM-01	Governance framework, ABAC model, roles
IAM-02	SSO, SAML 2.0, MFA
IAM-03	Group hierarchy, naming, SAML mapping
IAM-04	Policy syntax, conditions, patterns
IAM-05	Boundaries, three-domain model, buckets
IAM-06	SCIM, JIT, onboarding/offboarding, external users
IAM-07	Audit logs, SOC2/SOX/HIPAA/PCI-DSS
IAM-08	Multi-environment, break-glass, config sync
IAM-09	Troubleshooting methodology, diagnostic queries
IAM-10	Templated policies, `${bindParam:...}`, API bindings
IAM-12	API provisioning scripts, DQL validation

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.